Week No: 1	Roll Number: 23071A05B0	Date: 20|1|26
Week 1 Title: Data Handling Functions		

Aim: To implement Data Handling Functions.

Program/Output:

```
import pandas as pd
df=pd.read_csv("D:\\VJIET\\CSE\\data1.csv")
df
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger

```
df.shape
```

```
(398, 9)
```

```
df.columns
```

```
Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
       'acceleration', 'model year', 'origin', 'car name'],
      dtype='object')
```

```
df.tail()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

df.rename(columns={"model year":"year"})

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 9 columns

df['origin']

```
0       1
1       1
2       1
3       1
4       1
       ..
393     1
394     2
395     1
396     1
397     1
Name: origin, Length: 398, dtype: int64
```

#bind sets of rows of data frame
df1=df2=df
df3=pd.concat([df1,df2]).reset_index(drop=True)
df3

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
791	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
792	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
793	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
794	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
795	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

796 rows × 9 columns

```
#bind sets of columns of data frame
df1=df2=df
df3=pd.concat([df1,df2],axis=1).reset_index(drop=True)
df3
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	mpg	cylinders	displacement	horsepower	weight	acceleratio
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu	18.0	8	307.0	130	3504	12.
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320	15.0	8	350.0	165	3693	11.
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite	18.0	8	318.0	150	3436	11.
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst	16.0	8	304.0	150	3433	12.
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino	17.0	8	302.0	140	3449	10.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl	27.0	4	140.0	86	2790	15.
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup	44.0	4	97.0	52	2130	24.
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage	32.0	4	135.0	84	2295	11.
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger	28.0	4	120.0	79	2625	18.
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10	31.0	4	119.0	82	2720	19.

398 rows × 18 columns

```
#find missing values
df.isnull()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
393	False	False	False	False	False	False	False	False	False
394	False	False	False	False	False	False	False	False	False
395	False	False	False	False	False	False	False	False	False
396	False	False	False	False	False	False	False	False	False
397	False	False	False	False	False	False	False	False	False

398 rows × 9 columns

df.isnull().sum()

```
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64
```

Week No: 2	Roll Number: 23071A05B0	Date: 27|1|26
Week 2 Title: Basic Statistical Functions for Data Exploration		

Aim: To implement Basic Statistical Functions for Data Exploration

Program/Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statistics import median
df=pd.read_csv('E:\\VNR-VJIET\\DE\\datasets\\auto-mpg.csv')
df
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 9 columns

```
#mean of displacemnet
df['displacement'].mean()
```

```
193.42587939698493
```

```
#median of displacemnet
df['displacement'].median()
```

```
148.5
```

```
#mode of displacemnet
```

df['displacement'].mode()

```
0    97.0
dtype: float64
```

df['displacement'].value_counts()

```
97.0     21
98.0     18
350.0    18
318.0    17
250.0    17
         ..
104.0     1
110.0     1
130.0     1
111.0     1
144.0     1
Name: displacement, Length: 82, dtype: int64
```

Measure of dataspread, standard deviation, Quartiles, IQRD

df.describe()

	mpg	cylinders	displacement	weight	acceleration	model year	origin
count	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000
mean	23.514573	5.454774	193.425879	2970.424623	15.568090	76.010050	1.572864
std	7.815984	1.701004	104.269838	846.841774	2.757689	3.697627	0.802055
min	9.000000	3.000000	68.000000	1613.000000	8.000000	70.000000	1.000000
25%	17.500000	4.000000	104.250000	2223.750000	13.825000	73.000000	1.000000
50%	23.000000	4.000000	148.500000	2803.500000	15.500000	76.000000	1.000000
75%	29.000000	8.000000	262.000000	3608.000000	17.175000	79.000000	2.000000
max	46.600000	8.000000	455.000000	5140.000000	24.800000	82.000000	3.000000

df.std()

```
mpg                 7.815984
```

```
cylinders         1.701004
displacement    104.269838
weight          846.841774
acceleration      2.757689
model year        3.697627
origin            0.802055
dtype: float64
```

```
# finding variances of all columns
columns=['mpg', 'cylinders', 'displacement',  'weight',
      'acceleration', 'model year', 'origin']
print('Variances of each column')
for col in columns:
   print(df[col].var())
```

```
Variances of each column
61.089610774274405
2.8934154399199943
10872.199152247364
717140.9905256768
7.604848233611381
13.672442818627143
0.6432920268850575
```

```
# finding standard deviations of all columns
print('standard deviations of each column')
for col in columns:
   print(df[col].std())
```

```
standard deviations of each column
7.815984312565782
1.7010042445332094
104.26983817119581
846.8417741973271
2.7576889298126757
3.697626646732623
0.8020548777266163
```

```
#Quartile
import numpy as np
```

```
q1=np.percentile(df['displacement'],25,interpolation='midpoint')
q2=np.percentile(df['displacement'],50,interpolation='midpoint')
q3=np.percentile(df['displacement'],75,interpolation='midpoint')
IQR=q3-q1

print(q1)
104.5

print(q2)
148.5

print(q3)
262.0

print(IQR)
157.5

q11=df['displacement'].quantile(0.25)
q11
104.25

q33=df['displacement'].quantile(0.75)
q33
262.0
```

Week No: 3	Roll Number: 23071A05B0	Date: 3|2|26
Week 3 Title: Plotting Graphs		

Aim: To plot graphs to better visualise the dataset

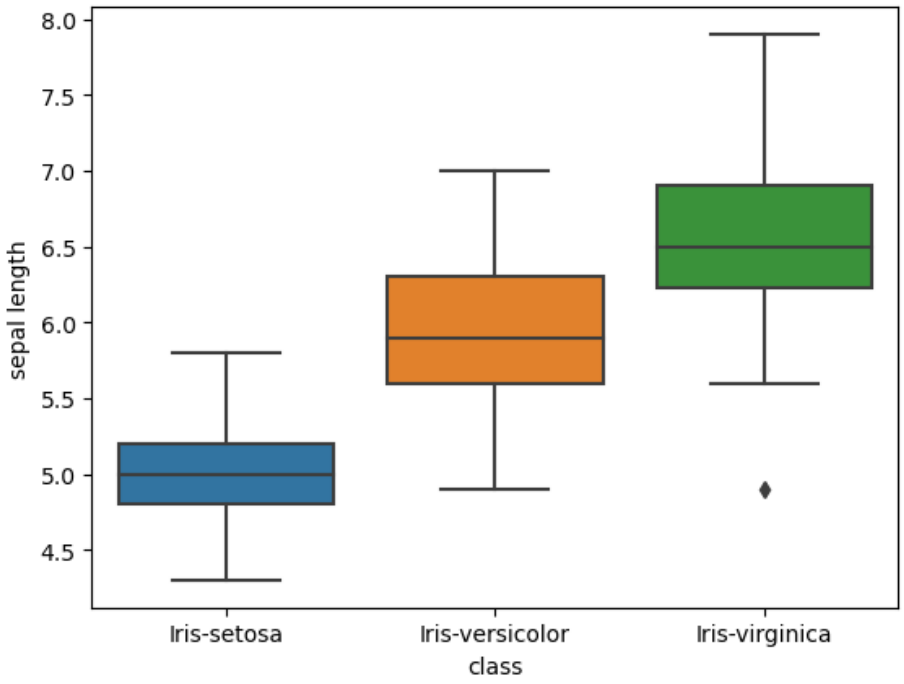
Program/Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statistics import median
df=pd.read_csv("C:\\iris.csv")
df
```

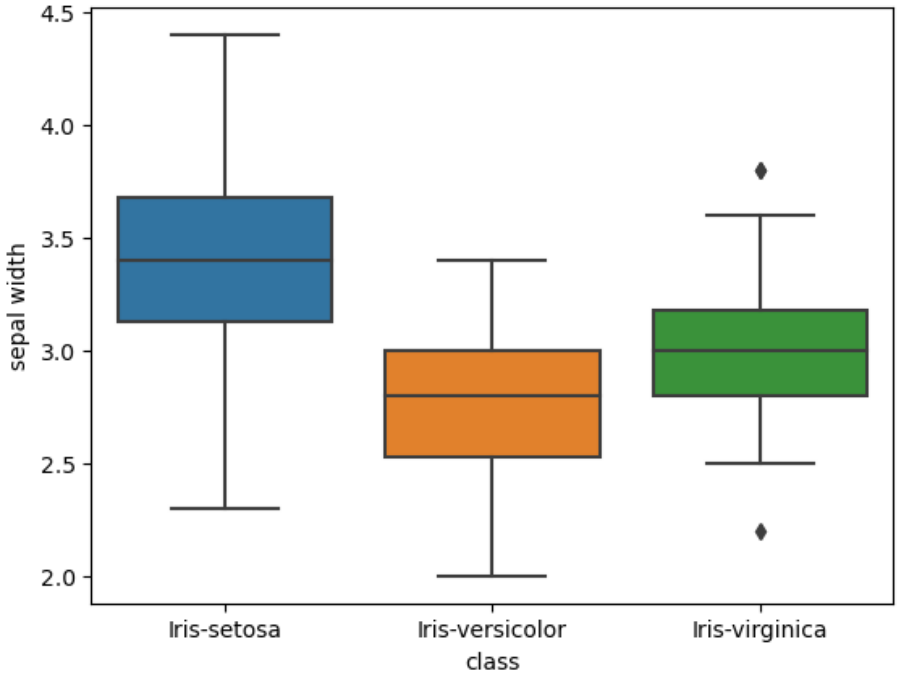
	sepal length	sepal width	petal length	petal width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

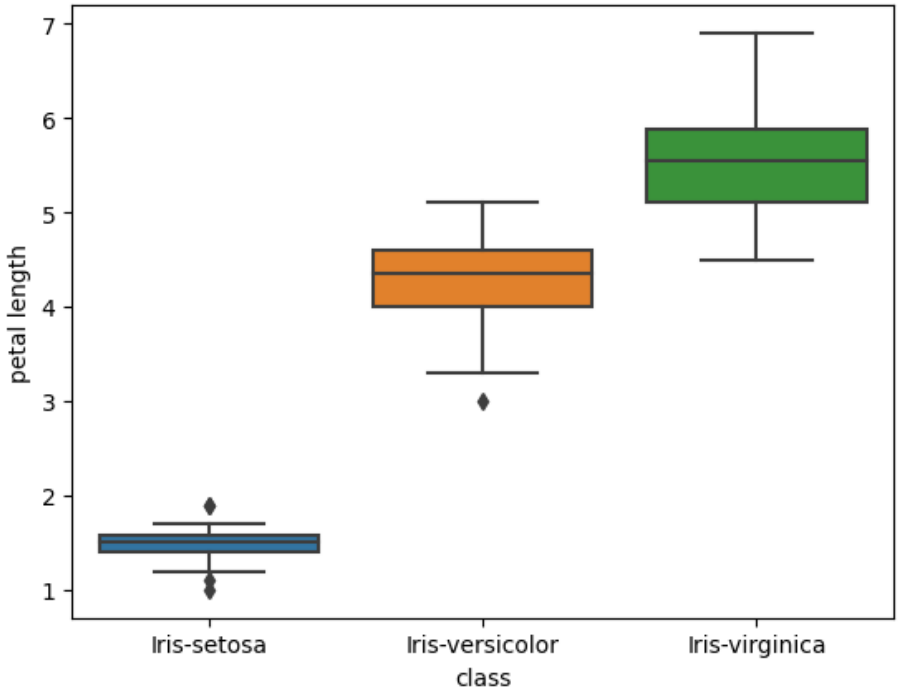
```
sns.boxplot(x='class',y='sepal length',data=df)
plt.show()
```



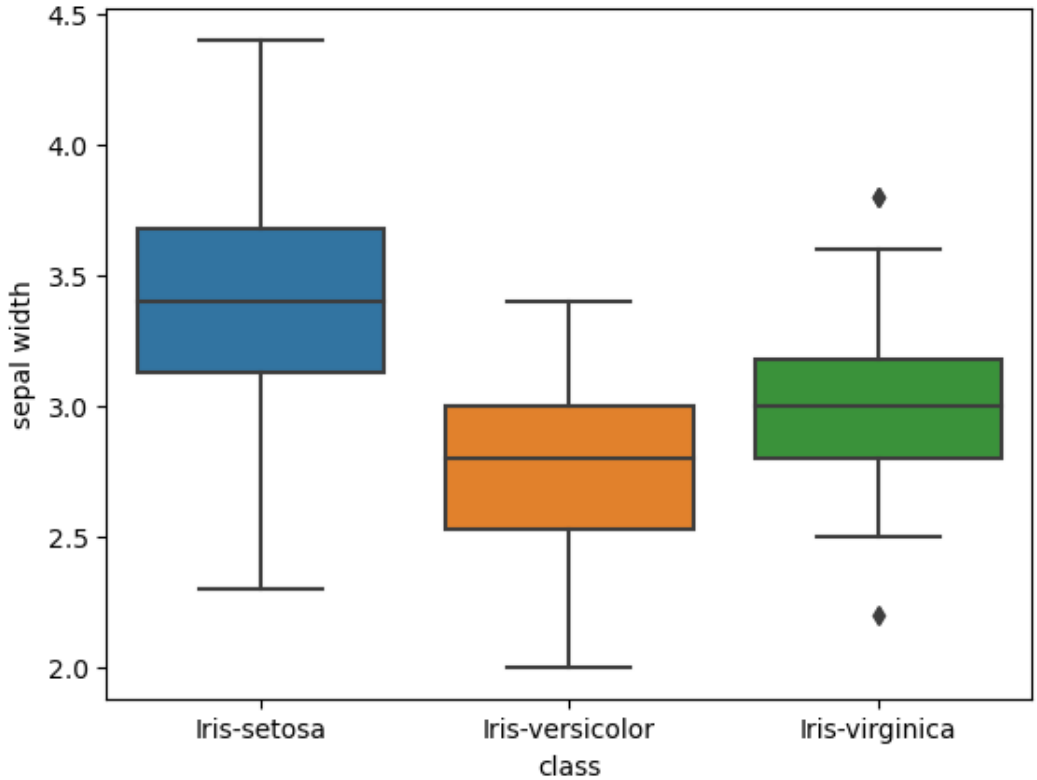

```
sns.boxplot(x='class',y='sepal width',data=df)
plt.show()
```




```
sns.boxplot(x='class',y='petal length',data=df)
plt.show()
```

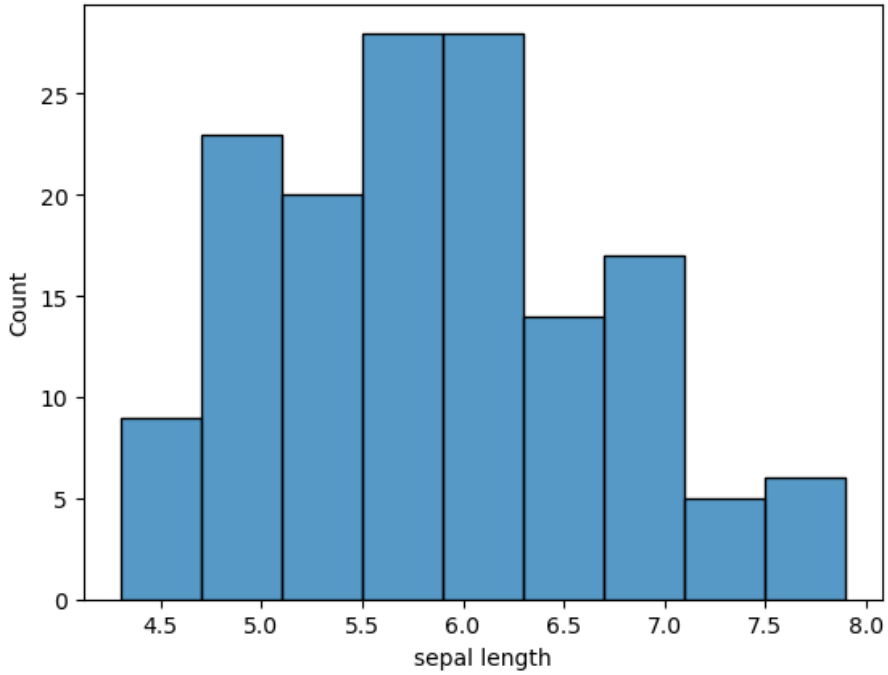



```
sns.boxplot(x='class',y='sepal width',data=df)
plt.show()
```



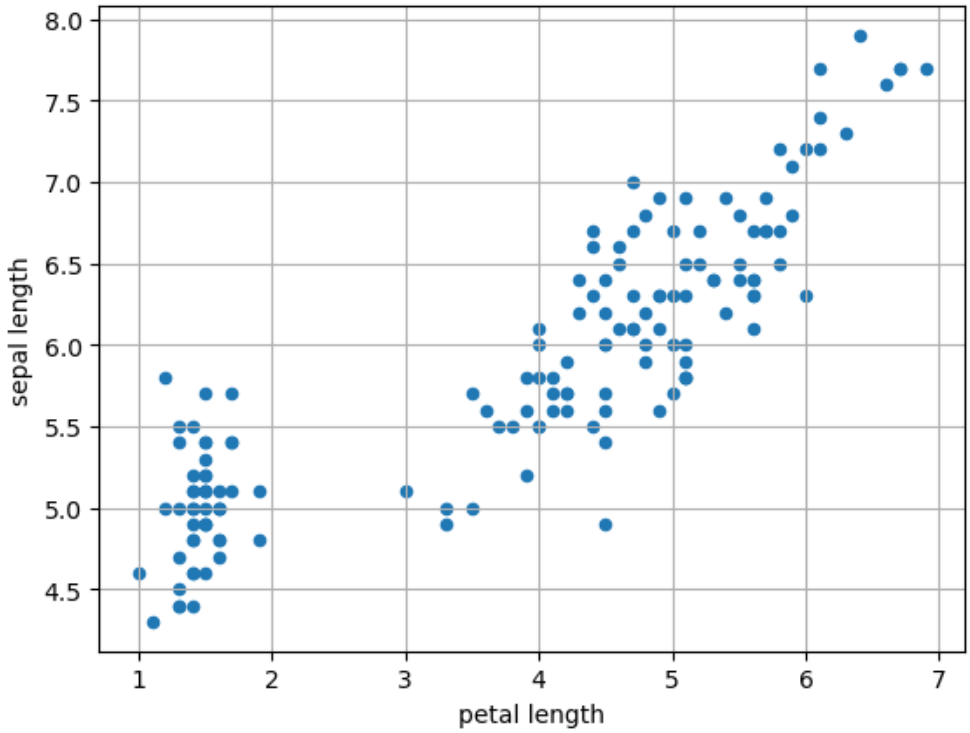

```
sns.histplot(x='sepal length',data=df)
```

plt.show()



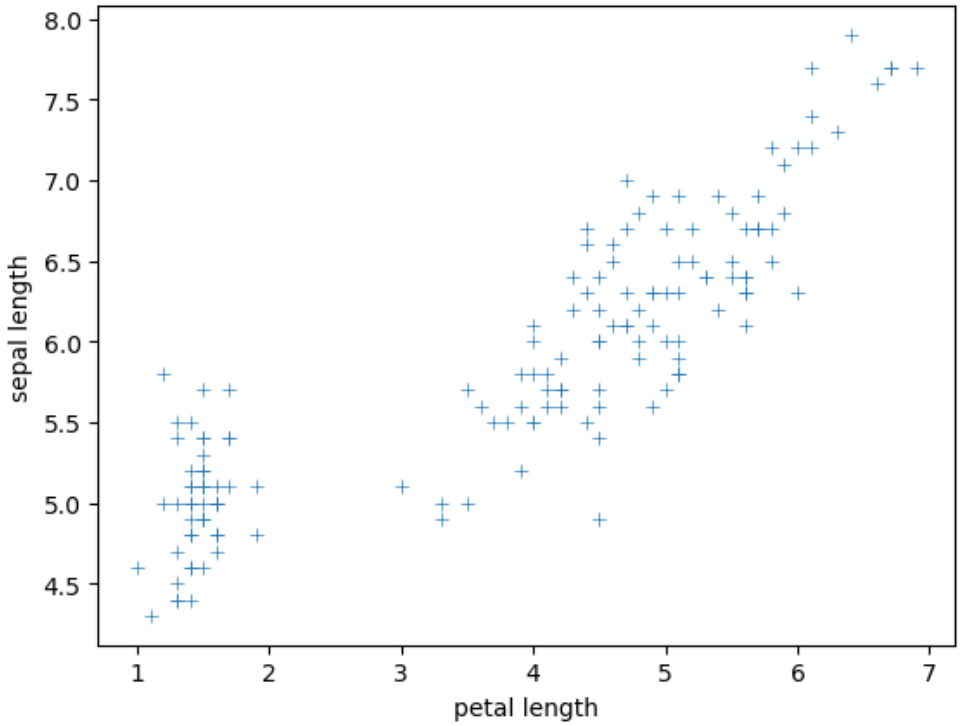
df.plot(kind='scatter',x='petal length',y='sepal length')

plt.grid()



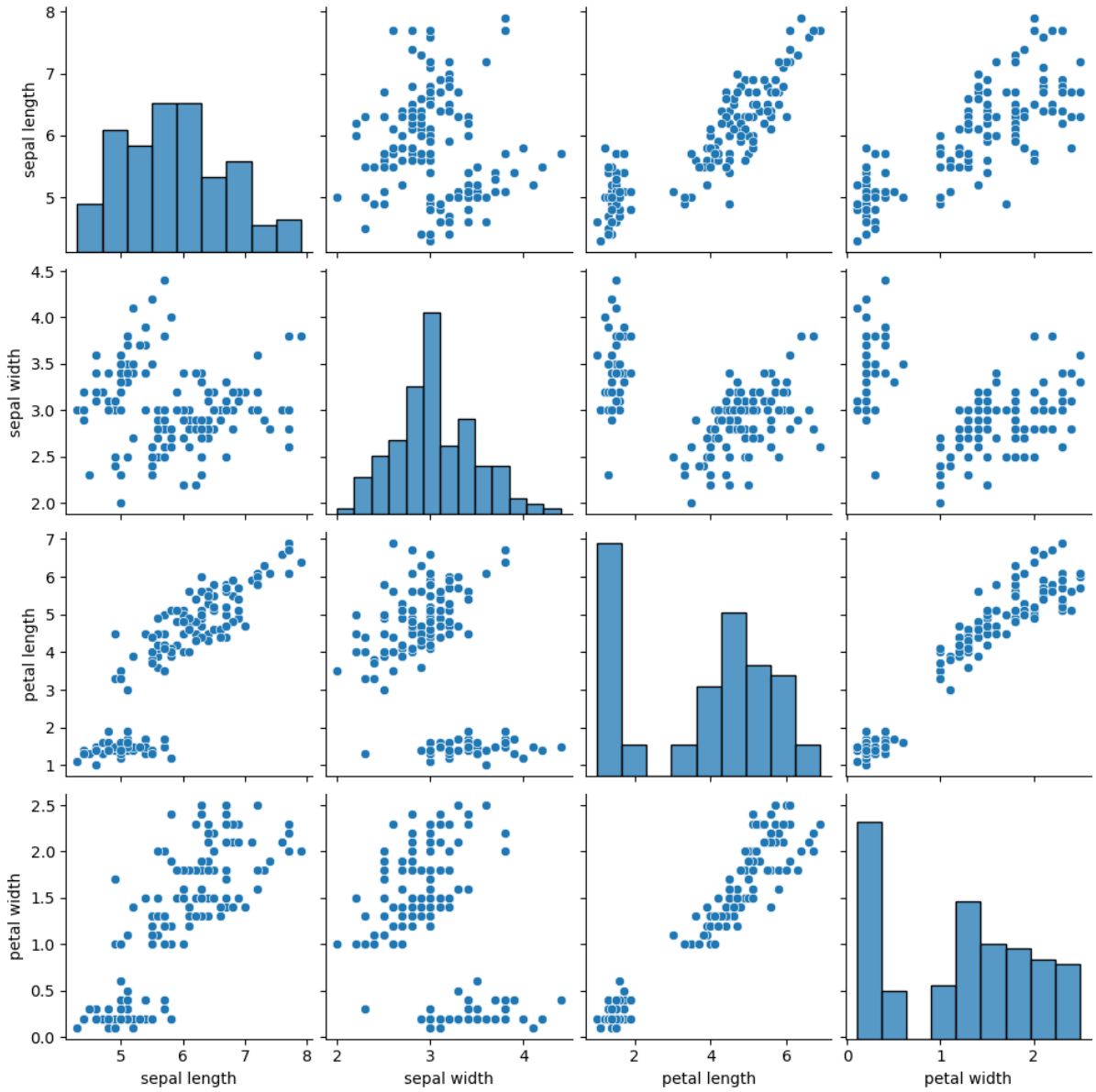
sns.scatterplot(x='petal length',y='sepal length',data=df,marker='+')

```
<Axes: xlabel='petal length', ylabel='sepal length'>
```



sns.pairplot(df)

```
<seaborn.axisgrid.PairGrid at 0x22ffa2af550>
```


sepal length
sepal width
petal length
petal width
sepal length
sepal width
petal length
petal width

Week No: 4	Roll Number: 23071A05B0	Date: 3|2|26
Week 4 Title: Data Preprocessing Techniques		

Aim: To implement Data preprocessing Techniques

Program/Output:

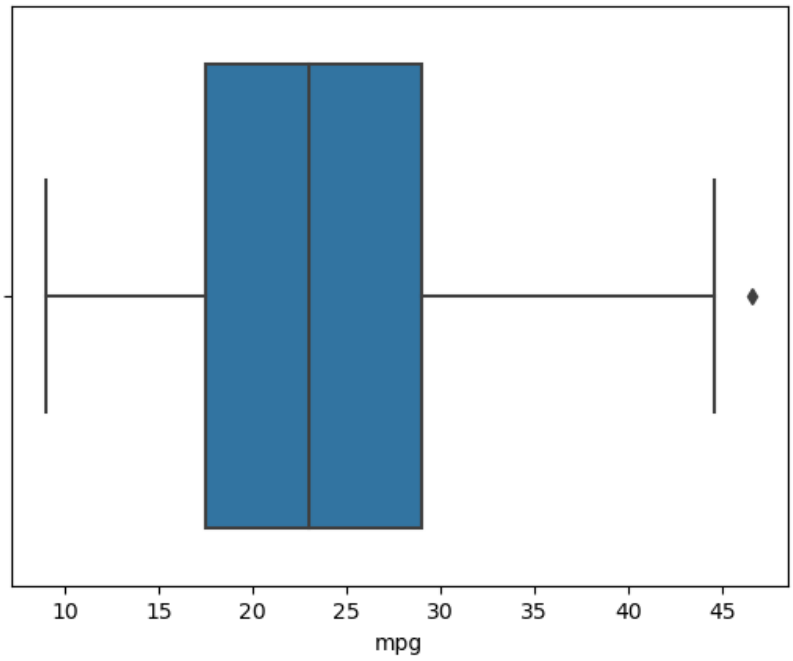
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statistics import median
import numpy as np
df=pd.read_csv('D:\\VJIET2022\\DE Lab_2-2-2023-24\\AIML C\\auto-mpg.csv')
df
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 9 columns

```
sns.boxplot(x='mpg',data=df)
```

```
<Axes: xlabel='mpg'>
```



dummy=df.loc[df['horsepower']=='?']

dummy

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
32	25.0	4	98.0	?	2046	19.0	71	1	ford pinto
126	21.0	6	200.0	?	2875	17.0	74	1	ford maverick
330	40.9	4	85.0	?	1835	17.3	80	2	renault lecar deluxe
336	23.6	4	140.0	?	2905	14.3	80	1	ford mustang cobra
354	34.5	4	100.0	?	2320	15.8	81	2	renault 18i
374	23.0	4	151.0	?	3035	20.5	82	1	amc concord dl

converting a column into an array

arr= df['horsepower'].values

arr

```
array(['130', '165', '150', '150', '140', '198', '220', '215', '225',
       '190', '170', '160', '150', '225', '95', '95', '97', '85', '88',
       '46', '87', '90', '95', '113', '90', '215', '200', '210', '193',
       '88', '90', '95', '?', '100', '105', '100', '88', '100', '165',
...
       '85', '92', '112', '96', '84', '90', '86', '52', '84', '79', '82'],
      dtype=object)
```

df.isnull().sum()

```
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64
```

# replace "?" to NaN
df.replace("?", np.nan, inplace = True)
df.head(5)

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

# calculate q1 (first quartile) and Q3(Third quartile).
q1=df.mpg.quantile(0.25)
q3=df.mpg.quantile(0.75)
#Calculate IQR
IQR=q3-q1
IQR

```
11.5
```

#calculate lowerlimit and upper limit
LL=q1-1.5*IQR
UL=q3+1.5*IQR
UL

```
46.25
```

LL

```
0.25
```

#Upper bound

```
upper = np.where(df['mpg'] >= UL)
upper
(array([322], dtype=int64),)

#lower bound
lower = np.where(df['mpg']<=LL)
lower
(array([], dtype=int64),)

# delete the rows above upper and below lower values
df.drop(upper[0], inplace=True)
df
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

397 rows × 9 columns

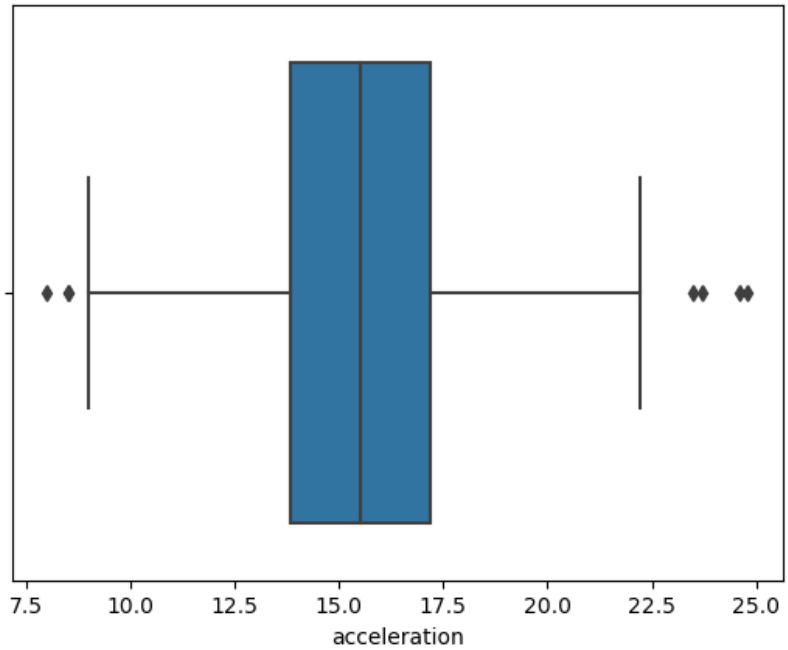
```
df.drop(lower[0], inplace=True)
df
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

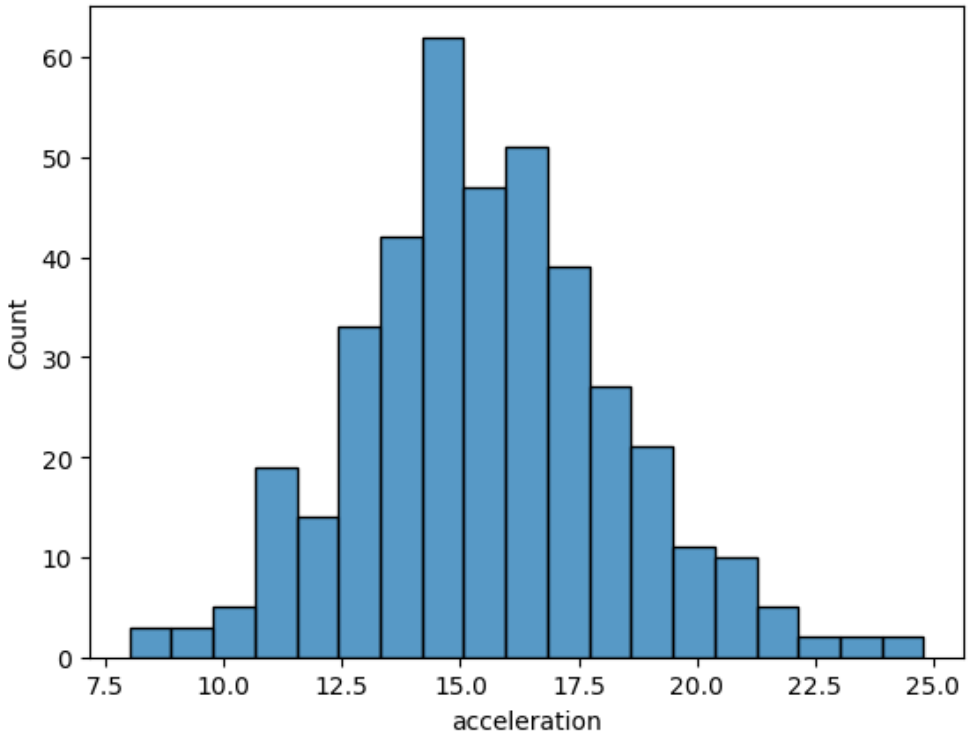
397 rows × 9 columns

```
sns.boxplot(x='acceleration',data=df2)
```

```
<Axes: xlabel='acceleration'>
```

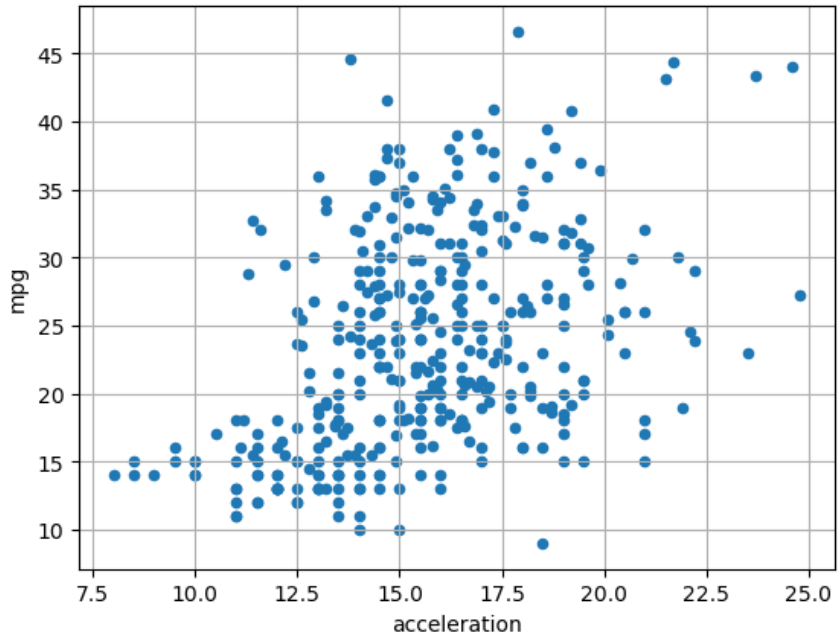


# Generate histogram for a specific feature
sns.histplot(x='acceleration',data=df2)
plt.show()



#Scatter plot

```
df.plot(kind="scatter",x='acceleration',y='mpg')
#plt.show()
plt.grid()
```




```
# IQR=q3-q1
q1=df2.acceleration.quantile(0.25)
q3=df2.acceleration.quantile(0.75)
IQR=q3-q1
IQR
LL=q1-1.5*IQR
UL=q3+1.5*IQR
UL
22.2

LL
8.8

upper = np.where(df2['acceleration'] > UL)
upper
(array([ 59, 299, 326, 394], dtype=int64),)



print(df2.loc[[326]])
     mpg  cylinders  displacement horsepower  weight  acceleration  \
```

```
326  43.4          4          90.0         48    2335          23.7

     model year  origin            car name
326          80       2  vw dasher (diesel)
```

lower = np.where(df2['acceleration'] < LL)

lower

```
(array([ 7,  9, 11], dtype=int64),)
```

med=df2['acceleration'].quantile(0.50)

med

```
15.5
```

indexing method

first convert the target values into an array

arr= df2['acceleration'].values

arr

```
array([12. , 11.5, 11. , 12. , 10.5, 10. ,  9. ,  8.5, 10. ,  8.5, 10. ,
        8. ,  9.5, 10. , 15. , 15.5, 15.5, 16. , 14.5, 20.5, 17.5, 14.5,
...
       16.2, 16.4, 17. , 14.5, 14.7, 13.9, 13. , 17.3, 15.6, 24.6, 11.6,
       18.6, 19.4])
```

if arr values are in between ll and ul, represent that

value as True else as False

False values are outliers we can find it at 5th position

true_index = (arr>LL) & (arr < UL)

true_index

Outliers are represented as False

```
rray([ True,  True,  True,  True,  True,  True,  True, False,  True,
       False,  True, False,  True,  True,  True,  True,  True,  True,
...
        True,  True,  True,  True,  True,  True,  True, False,  True,
        True,  True])
```

let us substitute mean value in the place of outliers

so ca1culate mean of all values other than the outliers.

mid1=np.mean(df2['acceleration'][true_index])

mid1

```
15.501542416452441
```

#replace False index value by mean
false_index= ~true_index # ~ is bool operator for inverse
df2['acceleration'].values[false_index]=mid1
df2

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.000000	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.500000	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.000000	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.000000	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.500000	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.600000	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	15.501542	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.600000	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.600000	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.400000	82	1	chevy s-10

398 rows × 9 columns

print(np.where(df2['acceleration'] > UL))

```
(array([], dtype=int64),)
```

df1=pd.read_csv('D:\\VJIET2022\\DE Lab_2-2-2023-24\\AIML C\\auto-mpg.csv')
df1

	Unnamed: 0	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	0	18.0	8	307.0	130	3504	12.000000	70	1	chevrolet chevelle malibu
1	1	15.0	8	350.0	165	3693	11.500000	70	1	buick skylark 320
2	2	18.0	8	318.0	150	3436	11.000000	70	1	plymouth satellite
3	3	16.0	8	304.0	150	3433	12.000000	70	1	amc rebel sst
4	4	17.0	8	302.0	140	3449	10.500000	70	1	ford torino
...	...	...	...	...	...	...	...	...	...	...
393	393	27.0	4	140.0	86	2790	15.600000	82	1	ford mustang gl
394	394	44.0	4	97.0	52	2130	15.501542	82	2	vw pickup
395	395	32.0	4	135.0	84	2295	11.600000	82	1	dodge rampage
396	396	28.0	4	120.0	79	2625	18.600000	82	1	ford ranger
397	397	31.0	4	119.0	82	2720	19.400000	82	1	chevy s-10

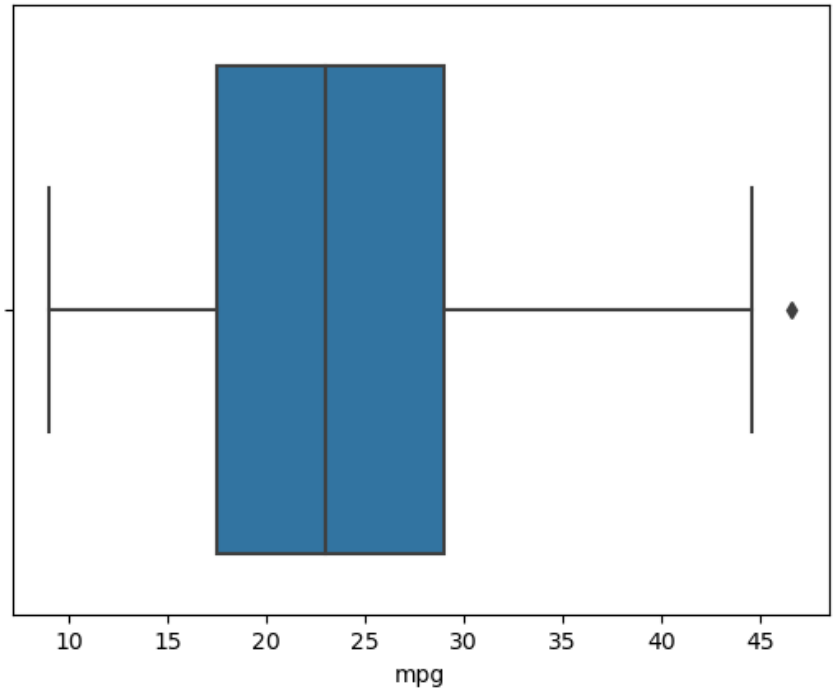
398 rows × 10 columns

df1.shape

```
(398, 10)
```

sns.boxplot(x='mpg',data=df1)

```
<Axes: xlabel='mpg'>
```



#Identify outliers in cylinders

max_threshold=df1['acceleration'].quantile(0.95)

min_threshold=df1['acceleration'].quantile(0.05)

max_threshold

```
19.514999999999997
```

min_threshold

```
11.485
```

displaying outlier index values

upper[0]

```
array([ 59, 299, 326, 394], dtype=int64)
```

#displaying outlier from dataest using index value

print(df1.loc[[59]])

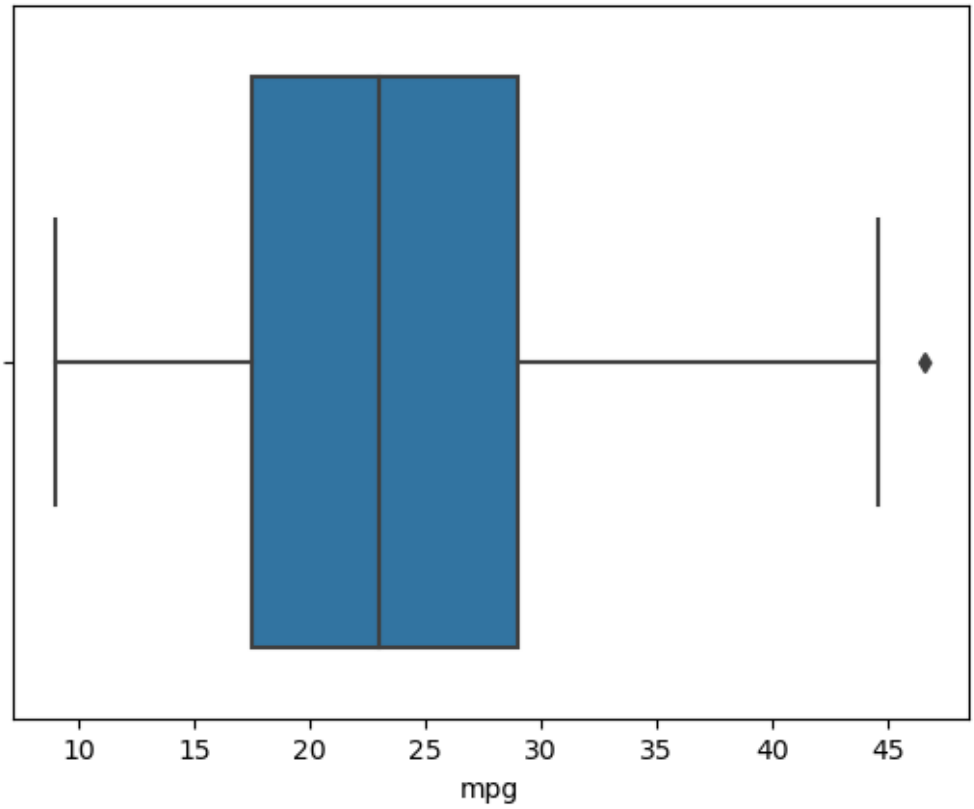
```
   Unnamed: 0   mpg  cylinders  displacement horsepower  weight  \
59          59  23.0          4          97.0         54    2254
```

Week No: 5	Roll Number: 23071A05B0	Date: 3|2|26
Week 5 Title: Feature Construction		

```
    acceleration  model year  origin           car name
59     15.501542          72       2  volkswagen type 3
```

sns.boxplot(x='mpg',data=df1)

```
<Axes: xlabel='mpg'>
```



Aim: To implement Feature Construction Techniques

Program/Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv('D:\\VJIET2022\\DE Lab_3-2-2026\\CSE B\\homeprices.csv')
df
```

	town	area	price
0	monroe township	2600	550000
1	monroe township	3000	565000
2	monroe township	3200	610000
3	monroe township	3600	680000
4	monroe township	4000	725000
5	west windsor	2600	585000
6	west windsor	2800	615000
7	west windsor	3300	650000
8	west windsor	3600	710000
9	robinsville	2600	575000
10	robinsville	2900	600000
11	robinsville	3100	620000
12	robinsville	3600	695000

```
#1. Dummy coding categorical(nominal) variables.
#Using pandas to create dummy variables
dummies = pd.get_dummies (df.town)
dummies
```

	monroe township	robinsville	west windsor
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0
5	0	0	1
6	0	0	1
7	0	0	1
8	0	0	1
9	0	1	0
10	0	1	0
11	0	1	0
12	0	1	0

```
#to avoid dummpy variable trap, remove one column
dummies = dummies.drop(['west windsor'], axis='columns')
dummies
```

	monroe township	robinsville
0	1	0
1	1	0
2	1	0
3	1	0
4	1	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	1
10	0	1
11	0	1
12	0	1

```
#add these dummies to original df. add olumns of ho.
merged = pd.concat ([df, dummies], axis='columns')
#we do dummy var not require 'town variable as Tt Is replaced h
#Hence drop town.
final= merged.drop(['town'], axis='columns')
final
```

	area	price	monroe township	robinsville
0	2600	550000	1	0
1	3000	565000	1	0
2	3200	610000	1	0
3	3600	680000	1	0
4	4000	725000	1	0
5	2600	585000	0	0
6	2800	615000	0	0
7	3300	650000	0	0
8	3600	710000	0	0
9	2600	575000	0	1
10	2900	600000	0	1
11	3100	620000	0	1
12	3600	695000	0	1

```
#Another method of encoding categorical(nominal) variables using "one hot encoding"
from sklearn.preprocessing import OneHotEncoder
ohe= OneHotEncoder (handle_unknown="ignore")
X1=ohe. fit_transform(df[['town']])
X1=pd.DataFrame (X1.toarray())
#Droping one column
X1 = X1.iloc[:, 1:]
X1
```

	1	2
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	1.0
6	0.0	1.0
7	0.0	1.0
8	0.0	1.0
9	1.0	0.0
10	1.0	0.0
11	1.0	0.0
12	1.0	0.0

#Encoding categorical(ordinal) variables(Using Label encoder)

```
import pandas as pd
iris= pd.read_csv('D:\\VJIET2022\\DE Lab_2-2-2023-24\\AIML C\\Iris.csv')
iris
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

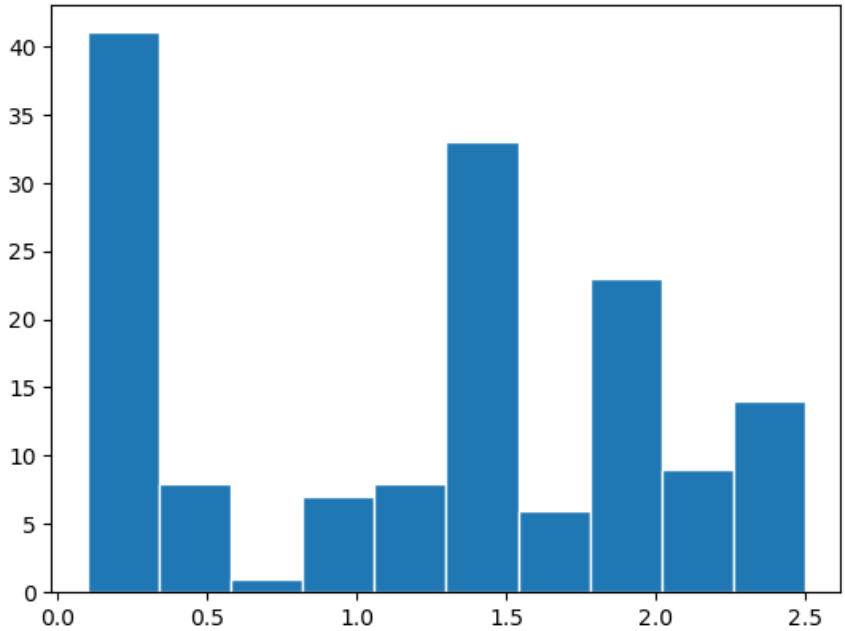
#Label encoding of nominal data using sklearn

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
iris_1=iris
iris_1.Species=le.fit_transform(iris_1.Species)
iris_1
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	0
1	2	4.9	3.0	1.4	0.2	0
2	3	4.7	3.2	1.3	0.2	0
3	4	4.6	3.1	1.5	0.2	0
4	5	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	2
146	147	6.3	2.5	5.0	1.9	2
147	148	6.5	3.0	5.2	2.0	2
148	149	6.2	3.4	5.4	2.3	2
149	150	5.9	3.0	5.1	1.8	2

150 rows × 6 columns

```
#3. Transforming numeric(continuous)features to categorical features
import matplotlib.pyplot as plt
plt.hist(iris.PetalWidthCm,edgecolor="w")
plt.show()
```



iris['petal_category']=pd.cut(iris.PetalWidthCm ,bins=[0,1,2,2.5])

print(iris['petal_category'])

```
0       (0.0, 1.0]
1       (0.0, 1.0]
2       (0.0, 1.0]
3       (0.0, 1.0]
4       (0.0, 1.0]
           ...
145     (2.0, 2.5]
146     (1.0, 2.0]
147     (1.0, 2.0]
148     (2.0, 2.5]
149     (1.0, 2.0]
Name: petal_category, Length: 150, dtype: category
Categories (3, interval[float64, right]): [(0.0, 1.0] < (1.0, 2.0] < (2.0, 2.5]]
```

print(iris['petal_category'].value_counts())

```
(1.0, 2.0]    70
(0.0, 1.0]    57
(2.0, 2.5]    23
Name: petal_category, dtype: int64
```

#2. Encoding categorical(ordinal) variables

```
from sklearn.preprocessing import OrdinalEncoder
encoder=OrdinalEncoder()
iris['petal_category']=encoder.fit_transform(iris[['petal_category']])
iris
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	petal_category
0	1	5.1	3.5	1.4	0.2	0	0.0
1	2	4.9	3.0	1.4	0.2	0	0.0
2	3	4.7	3.2	1.3	0.2	0	0.0
3	4	4.6	3.1	1.5	0.2	0	0.0
4	5	5.0	3.6	1.4	0.2	0	0.0
...	...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	2	2.0
146	147	6.3	2.5	5.0	1.9	2	1.0
147	148	6.5	3.0	5.2	2.0	2	1.0
148	149	6.2	3.4	5.4	2.3	2	2.0
149	150	5.9	3.0	5.1	1.8	2	1.0

150 rows × 7 columns

Week No: 6	Roll Number: 23071A05B0	Date: 3|2|26
Week 6 Title: Feature Construction		

Aim: To implement Feature Construction Techniques

Program/Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.decomposition import PCA
import sklearn.preprocessing
df=datasets.load_iris()
df
```

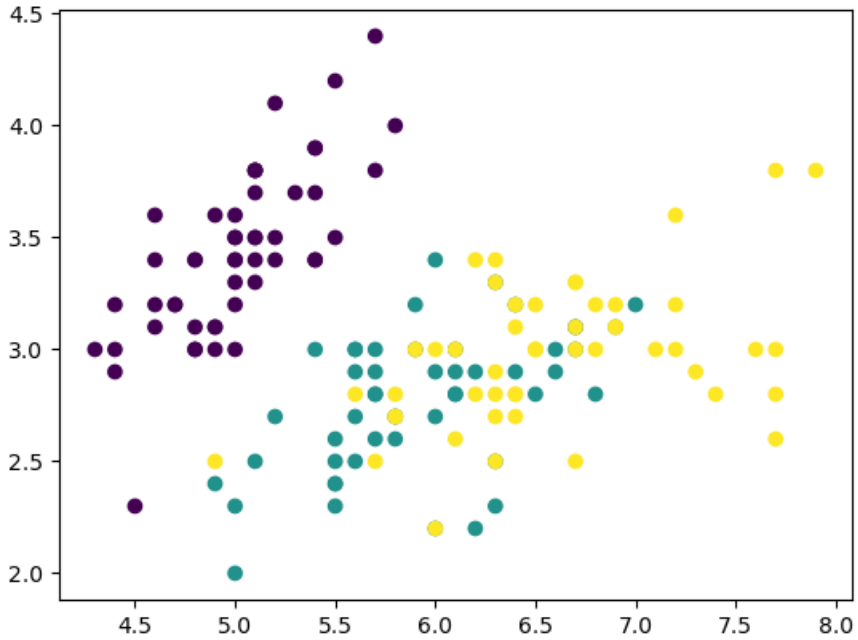
```
{'data': array([[5.1, 3.5, 1.4, 0.2],
        [4.9, 3. , 1.4, 0.2],
...
        [6.5, 3. , 5.2, 2. ],
        [6.2, 3.4, 5.4, 2.3],
        [5.9, 3. , 5.1, 1.8]]),
```

```
#1. Principal Component Analysis (PCA)
#Determining initial dimensions of dataset
x=df.data
y=df.target
print(x.shape,y.shape)
```

```
(150, 4) (150,)
```

```
#scatter plot
plt.scatter(x[:,0],x[:,1],c=y)
```

```
<matplotlib.collections.PathCollection at 0x1b825612f80>
```

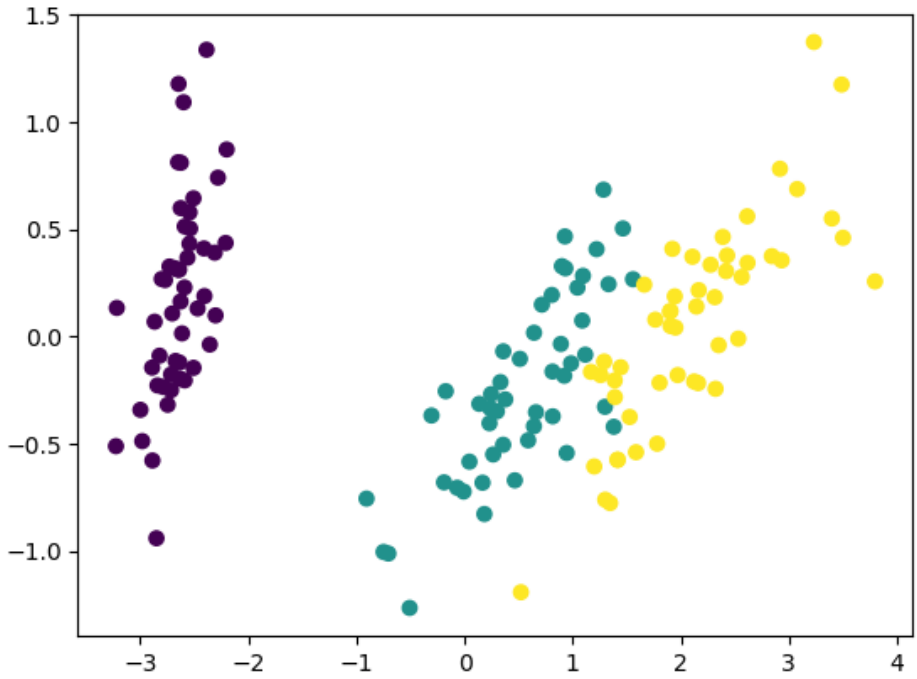



```
from sklearn.decomposition import PCA
pca = PCA()
X_new = pca.fit_transform(x)
cov_mat=pca.get_covariance()
import numpy as np
eig_vals, eig_vecs=np.linalg.eig(cov_mat)
#PCA for tagrget dimension on the dataset
pca=PCA(n_components=2)
pca.fit(x)
# Visualizing principle components
pca.components_
array([[ 0.36138659, -0.08452251,  0.85667061,  0.3582892 ],
       [ 0.65658877,  0.73016143, -0.17337266, -0.07548102]])
# Transforming the dataset from 4 dimensions into 2 dimension using PCA
z=pca.transform(x)
z.shape
(150, 2)
```

#scatter plot

plt.scatter(z[:,0],z[:,1],c=y)

```
<matplotlib.collections.PathCollection at 0x1b829284df0>
```



Variance ratio of the target dimensions

pca.explained_variance_ratio_

```
array([0.92461872, 0.05306648])
```

#0.4 Singular Value Decomposition

iris1 = sklearn.datasets.load_iris()

iris1.data.shape

```
(150, 4)
```

df_iris = pd.DataFrame(iris1.data, columns=iris1.feature_names)

df_iris.shape

```
(150, 4)
```

U_iris, S_iris, Vt_iris = np.linalg.svd(df_iris, full_matrices=False)

U_iris.shape

```
(150, 4)
```

S_iris

```
array([95.95991387, 17.76103366,  3.46093093,  1.88482631])
```

Vt_iris

```
array([[-0.75110816, -0.38008617, -0.51300886, -0.16790754],
       [ 0.2841749 ,  0.5467445 , -0.70866455, -0.34367081],
       [ 0.50215472, -0.67524332, -0.05916621, -0.53701625],
       [ 0.32081425, -0.31725607, -0.48074507,  0.75187165]])
```

#Befroe SVD scatter plot

x=iris1.data

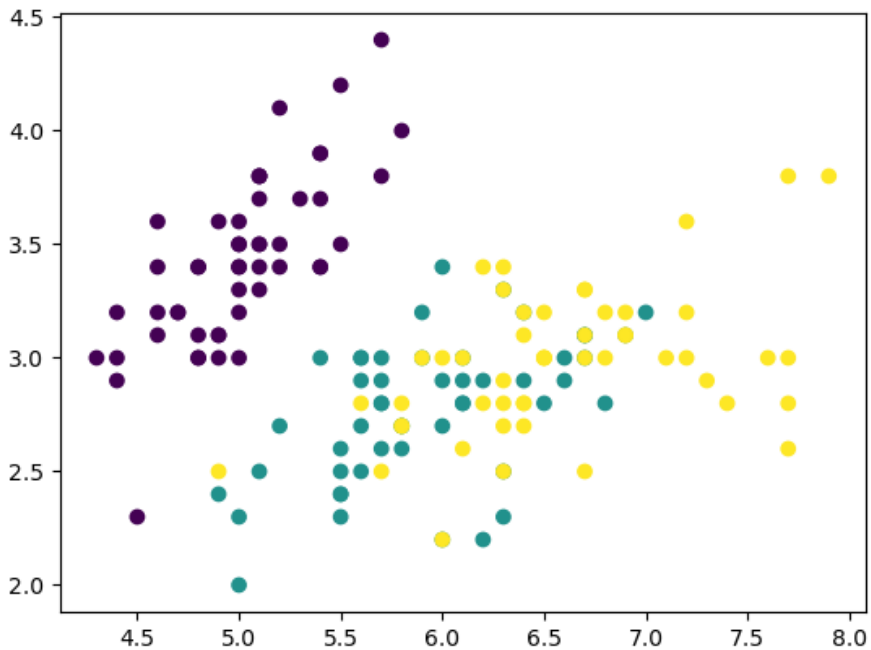
y=iris1.target

print(x.shape,y.shape)

#scatter plot

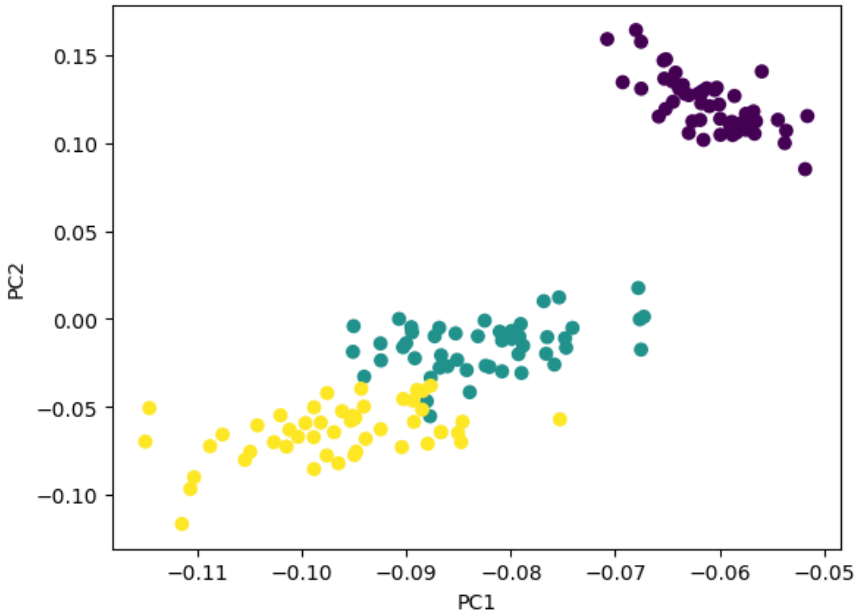
plt.scatter(x[:,0],x[:,1],c=y)

```
(150, 4) (150,)
<matplotlib.collections.PathCollection at 0x1b8292f37c0>
```



After SVD

```
import matplotlib.pyplot as plt
# Plot the first two columns of U
plt.scatter(U_iris[:, 0], U_iris[:, 1], c=iris1.target)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```



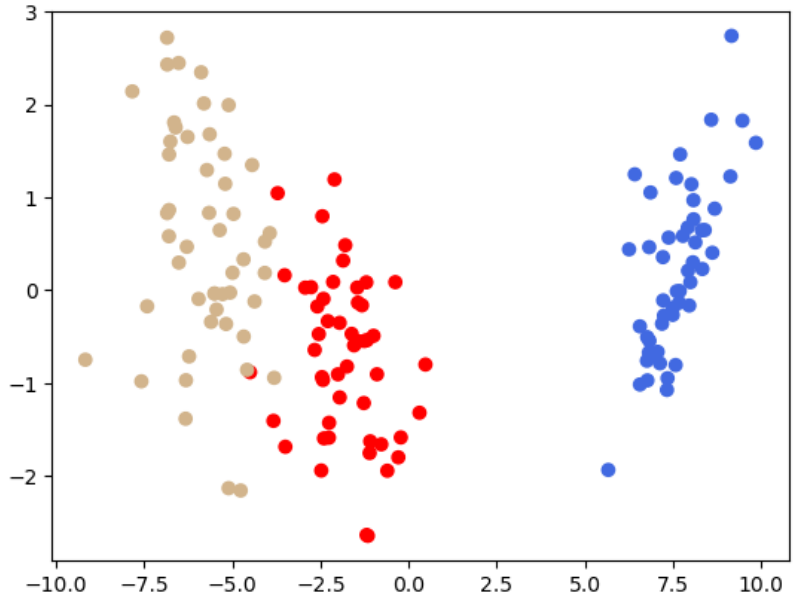

```
#3.Linear Discriminant Analysis (LDA)
#determing the initial dimensions of the dataset
X=df.data
Y=df.target
# DEcomposing 4D into 2D ising LDA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
lda=LinearDiscriminantAnalysis(n_components=2)
X_r2=lda.fit(X,Y).transform(X)
#Getting the variance ratio
lda.explained_variance_ratio_
```

```
array([0.9912126, 0.0087874])
```

```
# visualizing 2D datab in the form of scatter plot
```

```
import numpy as np
colors=['royalblue','red','tan']
vectorizer=np.vectorize(lambda X:colors[X%len(colors)])
plt.scatter(X_r2[:,0],X_r2[:,1],c=vectorizer(y))
```

```
<matplotlib.collections.PathCollection at 0x1b8293ffe50>
```



Feature Subset Selection

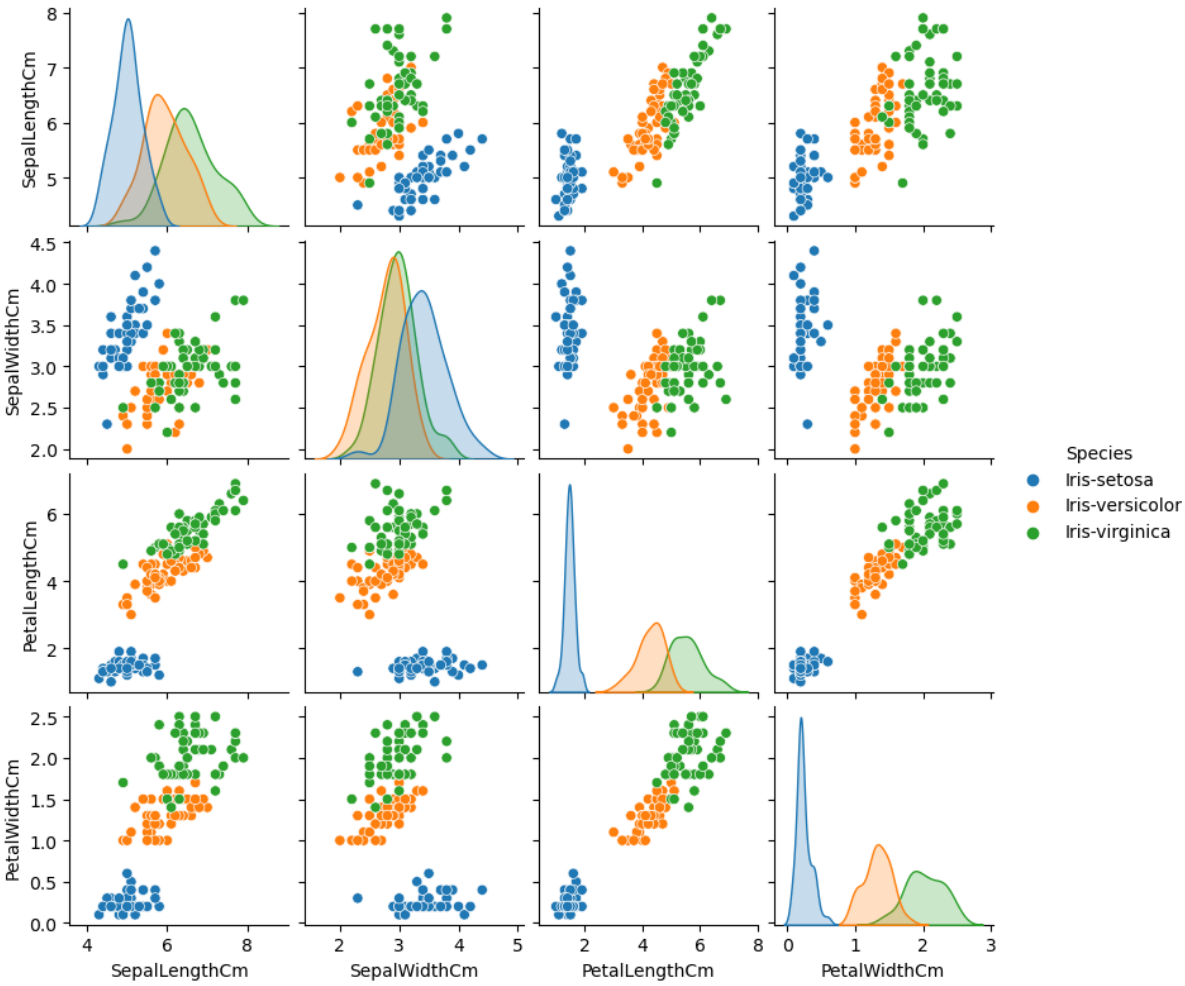
```
iris= pd.read_csv('D:\\VJIET2022\\DE Lab_2-2-2023-24\\AIML C\\Iris.csv')
iris
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

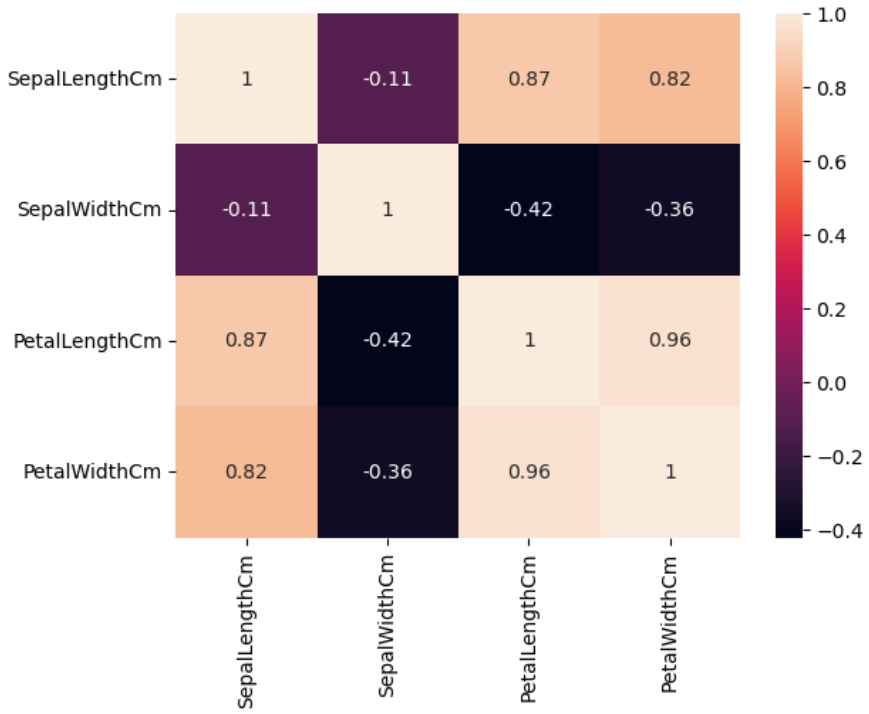
Visualizing using pair plot

```
sns.pairplot(iris.drop(['Id'],axis=1),hue='Species',height=2)
```




```
#Visualizing correlation using heatmap
```

```
sns.heatmap(iris.corr(method='pearson').drop(['Id'],axis=1).drop(['Id'],axis=0),annot=True)
plt.show()
```



Week No: 7	Roll Number: 23071A05B0	Date: 17|3|26
Week 7 Title: Hadoop Lab Experiment		

Aim: To learn basic HDFS commands for creating directories, transferring files between the local Linux system and HDFS, viewing file contents, and retrieving files from HDFS.

Program/Output:

Step 1: Start Hadoop Services

```
nihaldastagiri@Nihals-MacBook-Pro ~ % start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Nihals-MacBook-Pro.local]

nihaldastagiri@Nihals-MacBook-Pro ~ % start-yarn.sh
Starting resourcemanager
Starting nodemanagers

nihaldastagiri@Nihals-MacBook-Pro ~ % jps
47872 NodeManager
47443 DataNode
47940 Jps
47579 SecondaryNameNode
47771 ResourceManager
47340 NameNode
```

Step 2: Create Directory in HDFS

```
nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -mkdir /hadooplab

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -mkdir /hadooplab/input

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -ls /
Found 1 items
drwxr-xr-x   - nihaldastagiri supergroup          0 2026-03-30 16:56 /hadooplab
```

Step 3: Create Local File Using Linux Command

```
nihaldastagiri@Nihals-MacBook-Pro ~ % echo "Welcome to Hadoop Lab Experiment" >
sample.txt
nihaldastagiri@Nihals-MacBook-Pro ~ % cat sample.txt
```

Welcome to Hadoop Lab Experiment

Step 4: Upload File from Local System to HDFS

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -put sample.txt /hadooplab/input

Step 5: View Files in HDFS Directory

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -ls /hadooplab/input
Found 1 items
-rw-r--r-- 1 nihaldastagiri supergroup 33 2026-03-30 16:58 /hadooplab/input/sample.txt

Step 6: View File Contents Stored in HDFS

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -cat /hadooplab/input/sample.txt
Welcome to Hadoop Lab Experiment

Step 7: Retrieve File from HDFS to Local Disk

nihaldastagiri@Nihals-MacBook-Pro ~ % hdfs dfs -copyToLocal /hadooplab/input/sample.txt

Week No: 8 & 9	Roll Number: 23071A05B0	Date: 24|3|26
Week 8 & 9 Title: Map Reduce Programming		

Aim: To implement the word count program using hadoop and java

Program/Output:

Step 1: Creating directories

```
nihaldastagiri@Nihals-MacBook-Pro ~ % mkdir mapreduce
nihaldastagiri@Nihals-MacBook-Pro ~ % cd mapreduce
nihaldastagiri@Nihals-MacBook-Pro mapreduce % mkdir classes
```

Step 2: Make the WordCount.java file

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % nano WordCount.java
nihaldastagiri@Nihals-MacBook-Pro mapreduce % cat WordCount.java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper
      extends Mapper<Object, Text, Text, IntWritable> {

    private static final IntWritable one = new IntWritable(1);
    private Text word = new Text();
```

```
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

      StringTokenizer itr = new StringTokenizer(value.toString());

      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                Context context)
        throws IOException, InterruptedException {

      int sum = 0;

      for (IntWritable val : values) {
        sum += val.get();
      }

      result.set(sum);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
```

```
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Step 3: Compile the java file

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % javac -classpath `hadoop classpath` -d
classes WordCount.java
nihaldastagiri@Nihals-MacBook-Pro mapreduce % ls classes
WordCount.class
                                                                    WordCount$
IntSumReducer.class
                                                                    WordCount$
TokenizerMapper.class
```

Step 4: Create JAR file

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % jar -cvf wordcount.jar -C classes/ .
added manifest
adding: WordCount$IntSumReducer.class(in = 1793) (out= 776)(deflated 56%)
adding: WordCount$TokenizerMapper.class(in = 1790) (out= 793)(deflated 55%)
adding: WordCount.class(in = 1511) (out= 832)(deflated 44%)
nihaldastagiri@Nihals-MacBook-Pro mapreduce % jar -tf wordcount.jar
META-INF/
META-INF/MANIFEST.MF
WordCount$IntSumReducer.class
WordCount$TokenizerMapper.class
WordCount.class
```

Step 5: Create the input file

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % nano count.txt
```

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % cat count.txt
hello world hello hadoop mapreduce hadoop
```

Step 6: Run the MapReduce job

```
nihaldastagiri@Nihals-MacBook-Pro mapreduce % hadoop jar wordcount.jar WordCount
count.txt output
nihaldastagiri@Nihals-MacBook-Pro mapreduce % cat output/part-r-00000
hadoop                                                                  2
hello                                                                   2
mapreduce                                                               1
world                                                                   1
```

Week No: 10 & 11	Roll Number: 23071A05B0	Date: 24|3|26
Week 10 & 11 Title: Hive		

Aim: To understand Data Processing Tool – Hive and HQL (Hive query language)

Program/Output:

Step 1: Enter Hive Shell

nihaldastagiri@Nihals-MacBook-Pro ~ % docker exec -it hive-server /bin/bash

root@48d784308050:/opt# hive

Logging initialized using configuration in file:/opt/hive/conf/hive-log4j2.properties

hive>

Step 2: Create Database

```
hive> show databases;
OK
default
Time taken: 0.21 seconds
hive> create database emp_details;
hive> use emp_details;
hive> create database emp_details;
hive> use emp_details;
OK
Time taken: 0.08 seconds
```

Step 3: Create Managed Table

```
hive> create table emp(
    empno int,
    ename string,
    job string,
```

```
    sal int,
    deptno int
) row format delimited fields terminated by ',';
OK
Time taken: 0.32 seconds
```

Step 4: Load Data from Local

```
hive> LOAD DATA LOCAL INPATH '/root/empdetails.txt'
OVERWRITE INTO TABLE emp;
Loading data to table emp_details.emp
OK
Time taken: 0.45 seconds
```

Step 5: View Data

```
hive> select * from emp;
1   A   clerk   4000   10
2   A   clerk   4000   30
3   B   mgr     8000   20
4   C   peon    2000   40
5   D   clerk   4000   10
6   E   mgr     8000   50
Time taken: 0.27 seconds
```

Step 5: Describe Table

```
hive> describe emp;
empno    int
ename    string
```

job string

sal int

deptno int

Step 7: Show Tables

hive> show tables;

OK

emp

Time taken: 0.18 seconds

Step 8: Select ename

hive> select ename from emp;

A

A

B

C

D

E

Step 9: Filter Records

hive> select * from emp where ename='A';

1 A clerk 4000 10

2 A clerk 4000 30

Step 10: Count Records

hive> select count(*) from emp;

6

Step 11: Group By

hive> select deptno, sum(sal) from emp group by deptno;

10 8000

20 8000

30 4000

40 2000

50 8000

Step 12: Salary Range

hive> select * from emp where sal between 1000 and 2000;

4 C peon 2000 40

Step 13: LIKE Query

hive> alter table emp add columns(lastname string);

OK

Step 14: List employee names where job has 'l' as second character

hive> select ename from emp where job LIKE '_l%';

A

A

D

Step 15: Retrieve total salary for each department

hive> select deptno, sum(sal) from emp group by deptno;

10 8000

20 8000

30 4000

40 2000

50 8000

Step 16: Add Column

hive> alter table emp add columns(lastname string);

OK

Step 17: Rename Table

hive> alter table emp rename to emp1;

OK

Step 18: Drop Table

hive> drop table emp1;

OK

Step 19: External Table

hive> create external table ext1(a int, b int, c int)

row format delimited

fields terminated by ','

location '/root/hivedata';

OK

hive> select * from ext1;

1 2 3

4 5 6

Step 20: Movies Dataset

hive> create database movies;

hive> use movies;

```
hive> create table movie_details(
no int,
name string,
year int,
rating float,
views int
)
row format delimited
fields terminated by ',';
```

Step 21: Movie Queries

```
hive> select count(*) from movie_details;
100
hive> select max(rating) from movie_details;
9.5
```

PARTITIONING & BUCKETING

```
hive> create database shopping;
hive> use shopping;
```

Step 1: Create Table

```
hive> create table shopping1(
code int,
item_name string,
category string,
place string
)
row format delimited fields terminated by ',';
```

Step 2: Load Data

```
hive> LOAD DATA LOCAL INPATH '/root/shop.txt'
OVERWRITE INTO TABLE shopping1;
```

Step 3: Partition + Bucketing

```
hive> create table shopping3(
code int,
item_name string,
place string
)
partitioned by (category string)
clustered by (place) into 3 buckets
row format delimited
fields terminated by ','
stored as textfile;
```

Step 4: Insert Data

```
hive> from shopping1
insert overwrite table shopping3
partition(category)
select code, item_name, place, category;
OK
Time taken: 1.2 seconds
```

Step 5: Check Partitions (HDFS)

```
root@48d784308050:/opt# hadoop fs -ls /user/hive/warehouse/shopping.db/shopping3
Found 8 items
```

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=HIVE_DEFAULT_PARTITION

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=apparel

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=bag

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=cosmetic

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=electronicgadget

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=furnishing

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=toys

drwxrwxrwx - root supergroup 0 2026-04-14 12:10
/user/hive/warehouse/shopping.db/shopping3/category=utensils

Step 6: Check Buckets

root@48d784308050:/opt# hadoop fs -ls
/user/hive/warehouse/shopping.db/shopping3/category=utensils

000000_0

000001_0

000002_0

Week No: 12 & 13	Roll Number: 23071A05B0	Date: 31|3|26
Week 12 & 13 Title: MongoDB		

Aim: To understand MongoDB queries.

Program/Output:

Create a database and explore basic database commands

Create a database

```
test> use 23071A05B0
switched to db 23071A05B0
23071A05B0> db
23071A05B0
23071A05B0> show dbs
admin       40.00 KiB
config      36.00 KiB
local       72.00 KiB
myDatabase  64.00 KiB
23071A05B0> db.Student.insert({"Name":"Pushikara"})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('69de63fb54048a71e1f3417c') }
}
23071A05B0> show dbs
23071A05B0  40.00 KiB
admin       40.00 KiB
config      36.00 KiB
local       72.00 KiB
myDatabase  64.00 KiB
```

Drop Database

```
23071A05B0> db.dropDatabase()
{ ok: 1, dropped: '23071A05B0' }
23071A05B0> show dbs
admin       40.00 KiB
config      36.00 KiB
local       72.00 KiB
myDatabase  64.00 KiB
```

Step 3: Create Collection

```
23071A05B0> db.createCollection("Students")
{ ok: 1 }
23071A05B0> show collections
Students
```

Step 2: Drop Collection

```
23071A05B0> db.Students.drop()
true
23071A05B0> show collections
```

Create a database with sample dataset and work with MongoDB Query Language

Create Database:

```
23071A05B0> use emp
switched to db emp
```

Create Collection:

```
emp> db.createCollection("emp1")
{ ok: 1 }
```

Insert Records Into EMP1 Collection:

```
emp> db.emp1.insertMany([
|  {no:1,name:"ST",salary:2000,role:"OB"},
|  {no:2,name:"MSD",salary:1500,role:"WK"},
|  {no:3,name:"YS",salary:1000,role:"ALR"},
|  {no:4,name:"RD",salary:1000,role:"MOB"},
|  {no:5,name:"RS",salary:500,role:"OB"},
|  {no:6,name:"BK",salary:500,role:"MOB"},
|  {no:7,name:"VK",salary:300,role:"BW"},
|  {no:8,name:"JB",salary:400,role:"BW"},
|  {no:9,name:"HP",salary:400,role:"ALR"},
|  {no:10,name:"VS",salary:300,role:"OB"}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69de64de54048a71e1f3417d'),
    '1': ObjectId('69de64de54048a71e1f3417e'),
    '2': ObjectId('69de64de54048a71e1f3417f'),
    '3': ObjectId('69de64de54048a71e1f34180'),
    '4': ObjectId('69de64de54048a71e1f34181'),
    '5': ObjectId('69de64de54048a71e1f34182'),
    '6': ObjectId('69de64de54048a71e1f34183'),
    '7': ObjectId('69de64de54048a71e1f34184'),
```

```
    '8': ObjectId('69de64de54048a71e1f34185'),
    '9': ObjectId('69de64de54048a71e1f34186')
  }
}
```

Display Data in Proper Format:

```
emp> db.emp1.find().pretty()
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 2000,
    role: 'OB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f3417e'),
    no: 2,
    name: 'MSD',
    salary: 1500,
    role: 'WK'
  },
  {
    _id: ObjectId('69de64de54048a71e1f3417f'),
    no: 3,
    name: 'YS',
    salary: 1000,
    role: 'ALR'
  },
```

{
  _id: ObjectId('69de64de54048a71e1f34180'),
  no: 4,
  name: 'RD',
  salary: 1000,
  role: 'MOB'
},
{
  _id: ObjectId('69de64de54048a71e1f34181'),
  no: 5,
  name: 'RS',
  salary: 500,
  role: 'OB'
},
{
  _id: ObjectId('69de64de54048a71e1f34182'),
  no: 6,
  name: 'BK',
  salary: 500,
  role: 'MOB'
},
{
  _id: ObjectId('69de64de54048a71e1f34183'),
  no: 7,
  name: 'VK',
  salary: 300,
  role: 'BW'
},
{

```
    _id: ObjectId('69de64de54048a71e1f34184'),
    no: 8,
    name: 'JB',
    salary: 400,
    role: 'BW'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34185'),
    no: 9,
    name: 'HP',
    salary: 400,
    role: 'ALR'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34186'),
    no: 10,
    name: 'VS',
    salary: 300,
    role: 'OB'
  }
]
```

Update Salary Of Employee where Name is "ST" by +8000

```
emp> db.emp1.updateOne(
|   {name:"ST"},
|   {$inc:{salary:8000}}
| )
{
  acknowledged: true,
```

```
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Update Salary Of All Employee by giving an increment of +4000 each

```
emp> db.emp1.updateMany(
|   {},
|   {$inc:{salary:4000}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 10,
  modifiedCount: 10,
  upsertedCount: 0
}
```

Update role of "MSD" as "C and WK"

```
emp> db.emp1.updateOne(
|   {name:"MSD"},
|   {$set:{role:"C and WK"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
```

```
  upsertedCount: 0
}
```

Add a New Field remark to document with name "RS" set Remark as WC

```
emp> db.emp1.updateOne(
|   {name:"RS"},
|   {$set:{remark:"WC"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Add a New Field As Number 11,name AK, Salary 10000,role coch without using insert statement.

```
emp> db.emp1.updateOne(
|   {no:11},
|   {$set:{name:"AK",salary:10000,role:"coach"}},
|   {upsert:true}
| )
{
  acknowledged: true,
  insertedId: ObjectId('69de65baf25459187bffe5a8'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
```

Remove added New Field (REMARK)

```
emp> db.emp1.updateOne(
|   {name:"RS"},
|   {$unset:{remark:""}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Update the Field "RD" by multiplying with salary by 2

```
emp> db.emp1.updateOne(
|   {name:"RD"},
|   {$mul:{salary:2}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

To Find Document From the empl collection where name begins with S

```
emp> db.emp1.find({name:/^S/})
```

```
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 14000,
    role: 'OB'
  }
]
```

To Find Document From the empl collection where name begins with R

```
emp> db.emp1.find({name:/^R/})
[
  {
    _id: ObjectId('69de64de54048a71e1f34180'),
    no: 4,
    name: 'RD',
    salary: 10000,
    role: 'MOB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34181'),
    no: 5,
    name: 'RS',
    salary: 4500,
    role: 'OB'
  }
]
```

To Find Document From the empl collection where name ends with K

```
emp> db.emp1.find({name:/K$/})
[
  {
    _id: ObjectId('69de64de54048a71e1f34182'),
    no: 6,
    name: 'BK',
    salary: 4500,
    role: 'MOB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34183'),
    no: 7,
    name: 'VK',
    salary: 4300,
    role: 'BW'
  },
  {
    _id: ObjectId('69de65baf25459187bffe5a8'),
    no: 11,
    name: 'AK',
    role: 'coach',
    salary: 10000
  }
]
```

To Find Document From the empl collection where name ends with D

```
emp> db.emp1.find({name:/D$/})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417e'),
    no: 2,
    name: 'MSD',
    salary: 5500,
    role: 'C and WK'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34180'),
    no: 4,
    name: 'RD',
    salary: 10000,
    role: 'MOB'
  }
]
```

Find Document From the empl collection where name has S in any position

```
emp> db.emp1.find({name:/S/})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 14000,
    role: 'OB'
  },
```

```
  {
    _id: ObjectId('69de64de54048a71e1f3417e'),
    no: 2,
    name: 'MSD',
    salary: 5500,
    role: 'C and WK'
  },
  {
    _id: ObjectId('69de64de54048a71e1f3417f'),
    no: 3,
    name: 'YS',
    salary: 5000,
    role: 'ALR'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34181'),
    no: 5,
    name: 'RS',
    salary: 4500,
    role: 'OB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34186'),
    no: 10,
    name: 'VS',
    salary: 4300,
    role: 'OB'
  }
]
```

Using Regular Expression

Find Document From the empl collection where name begins with S

```
emp> db.emp1.find({name:{$regex:"^S"}})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 14000,
    role: 'OB'
  }
]
```

Find Document From the empl collection where name begins, ends and contains S

```
emp> db.emp1.find({name:{$regex:"S",$options:"i"}})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 14000,
    role: 'OB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f3417e'),
    no: 2,
```

```
  name: 'MSD',
  salary: 5500,
  role: 'C and WK'
 },
 {
  _id: ObjectId('69de64de54048a71e1f3417f'),
  no: 3,
  name: 'YS',
  salary: 5000,
  role: 'ALR'
 },
 {
  _id: ObjectId('69de64de54048a71e1f34181'),
  no: 5,
  name: 'RS',
  salary: 4500,
  role: 'OB'
 },
 {
  _id: ObjectId('69de64de54048a71e1f34186'),
  no: 10,
  name: 'VS',
  salary: 4300,
  role: 'OB'
 }
]
```

Display Documents where in empl collection Field have OB,MOB

```
emp> db.emp1.find({role:{$in:["OB","MOB"]}})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417d'),
    no: 1,
    name: 'ST',
    salary: 14000,
    role: 'OB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34180'),
    no: 4,
    name: 'RD',
    salary: 10000,
    role: 'MOB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34181'),
    no: 5,
    name: 'RS',
    salary: 4500,
    role: 'OB'
  },
  {
    _id: ObjectId('69de64de54048a71e1f34182'),
    no: 6,
    name: 'BK',
    salary: 4500,
    role: 'MOB'
```

```
  },
  {
    _id: ObjectId('69de64de54048a71e1f34186'),
    no: 10,
    name: 'VS',
    salary: 4300,
    role: 'OB'
  }
]
```

Display Documents where in empl collection Field not have OB,MOB

```
emp> db.emp1.find({role:{$nin:["OB","MOB"]}})
[
  {
    _id: ObjectId('69de64de54048a71e1f3417e'),
    no: 2,
    name: 'MSD',
    salary: 5500,
    role: 'C and WK'
  },
  {
    _id: ObjectId('69de64de54048a71e1f3417f'),
    no: 3,
    name: 'YS',
    salary: 5000,
    role: 'ALR'
  },
  {
```

```
  _id: ObjectId('69de64de54048a71e1f34183'),
  no: 7,
  name: 'VK',
  salary: 4300,
  role: 'BW'
 },
 {
  _id: ObjectId('69de64de54048a71e1f34184'),
  no: 8,
  name: 'JB',
  salary: 4400,
  role: 'BW'
 },
 {
  _id: ObjectId('69de64de54048a71e1f34185'),
  no: 9,
  name: 'HP',
  salary: 4400,
  role: 'ALR'
 },
 {
  _id: ObjectId('69de65baf25459187bffe5a8'),
  no: 11,
  name: 'AK',
  role: 'coach',
  salary: 10000
 }
]
```